UTN FRLP

Desarrollo de Software 2026

React: useContext

Clase #12 — 29 de junio de 2026

Matías Corti - Martina García Améndola



De dónde venimos

Las clases pasadas vimos…

- **C09:** estructura profesional, Layout y routing
- **C10:** formularios controlados + validación con Zod; la lista de libros vivió como estado en App.tsx
- **C11:** useFetch, fetch-per-view (cada vista pide su dato), patrón de 3 estados: data / loading / error
- **Hoy:** Cerramos el front con estado global, cuando un dato lo necesitan componentes lejanos entre sí.

Temas de hoy

- Qué es el estado global y por qué pasar props nivel por nivel duele (prop drilling)
- useContext: createContext, Provider, hook propio
- Qué SÍ y qué NO conviene poner en un context
- Cierre del Bloque 2: preparar la Entrega 2
- ⏱️ Clase corta: el resto del tiempo es para consultas y revisión por grupo



El problema: prop drilling

El problema:

Dos lugares lejanos del árbol necesitan el mismo dato.

- Ejemplo: un filtro de búsqueda. El Header (dentro del Layout) lo escribe; el Catálogo (otra página) lo usa.
- Para compartirlo por props hay que bajarlo por dos ramas distintas.

```
// App tiene el filtro... y lo baja a DOS ramas que no se conocen
function App() {
  const [filtro, setFiltro] = useState('');
  return (
    <Layout filtro={filtro} setFiltro={setFiltro}>  {/* solo para el Header */ }
      <Routes>
        <Route path="/catalogo" element={<Libros filtro={filtro} />} />
      </Routes>
    </Layout>
  );
}
// Layout NO usa el filtro: solo lo pasa al Header. Eso es "prop drilling".
```



¿Qué es useContext?

Es un hook que nos permite compartir un valor con todo un subárbol sin pasarlo prop por prop.

Tres piezas:

- createContext() → el contexto: es el "canal"
- Provider → el proveedor: dueño del estado, lo envuelve
- useContext() / hook propio → el consumidor: lo lee

✅ **Cuándo usarlo**	❌ **Cuándo NO usarlo**
Estado transversal entre componentes lejanos	Estado local de un componente
Filtro, tema, usuario logueado	Datos que pide una sola vista

1️⃣ CONTEXTO + PROVEEDOR

src/context/BusquedaContext.tsx

```tsx
import { createContext, useContext, useState } from 'react';
import type { ReactNode } from 'react';

interface BusquedaContextType {
  filtro: string;
  setFiltro: (valor: string) => void;
}

// 1. El contexto (arranca null: nadie lo usó todavía)
const BusquedaContext = createContext<BusquedaContextType |
null>(null);

// 2. El proveedor: tiene el estado y lo reparte
export function BusquedaProvider({ children }: { children: ReactNode })
{
  const [filtro, setFiltro] = useState('');
  return (
    <BusquedaContext.Provider value={{ filtro, setFiltro }}>
      {children}
    </BusquedaContext.Provider>
  );
}

// 3. Hook propio para consumirlo (con red de seguridad)
export function useBusqueda() {
  const contexto = useContext(BusquedaContext);
  if (!contexto) {
    throw new Error('useBusqueda debe usarse dentro de
<BusquedaProvider>');
  }
  return contexto;
}
```

[Ejemplo Completo: DS26-Libreria (C12)]

2

ENVOLVER + CONSUMIR

- Header y Catálogo no se conocen → el puente es el context
- El dato sigue viniendo del useFetch; context aporta solo el filtro
- El contador baja en vivo (10 → 3 → 0) mientras tipeás
- ⚠️ Filtramos en memoria: es una maqueta. Filtrado real va por API

```
// Header.tsx → ESCRIBE el filtro; al buscar, navega al catálogo
const { filtro, setFiltro } = useBusqueda();
const navigate = useNavigate();              // navegación programática
const buscar = (e) => { e.preventDefault(); navigate('/catalogo'); };

<Form onSubmit={buscar}>
  <InputGroup>                    {/* input + lupa, pegados */}
    <Form.Control value={filtro} placeholder="Buscar por título o
autor…"
                  onChange={e => setFiltro(e.target.value)} />
    <Button type="submit" variant="outline-secondary">🔍</Button>
  </InputGroup>
</Form>


// LibrosCatalogo.tsx → LEE el filtro (el dato sigue viniendo del useFetch)
const { data: libros } = useFetch<LibroCardProps[]>('/libros.json');
const { filtro } = useBusqueda();

// filtramos EN MEMORIA lo que ya trajimos → es una maqueta
const visibles = (libros ?? []).filter(l =>
 l.titulo.toLowerCase().includes(filtro.toLowerCase()));

// para que se note en vivo: contador + estado vacío
<p>Mostrando {visibles.length} de {(libros ?? []).length} libros</p>
{visibles.length === 0 && <Alert variant="secondary">Sin resultados
para "{filtro}"</Alert>}
```

Ejemplo Completo: DS26-Libreria (C12)



Arquitectura: qué va y qué no

Regla: context = estado de UI transversal y liviano.

Los datos del dominio viven en el backend, no en un context.

⚠️ **El filtro de hoy es una maqueta:** porque filtramos en memoria (todavía no hay backend).

Con backend irá por la API ('/libros?q=' + filtro) → el servidor devuelve ya filtrado.

✔ El **useContext** con el término se queda igual.

✅ **VA a context**	❌ **NO va a context**
Filtro de búsqueda local	Catálogo (cada vista)
Tema claro / oscuro	Formulario (local+RHF)
Usuario logueado	Alta/guardar (backend)

✅

Cierre del bloque técnico

- `useContext` en 3 pasos: `createContext` → `Provider` (dueño del estado) → hook para consumir
- Sirve cuando el estado lo comparten componentes lejanos
- No reemplaza props ni `useFetch`: convive con ellos
- **Regla de oro:** UI transversal sí, datos del dominio NO
- El `.filter()` de hoy es maqueta → con backend la búsqueda va por API, el context se queda

🎓 Con esto cerramos el contenido nuevo de Frontend. De acá en más: pulir y entregar.



Preparándose para la Entrega 2

RECORDATORIO
El **05/07/2026** es la **ENTREGA 2 (Frontend)** del **Trabajo Práctico Grupal**

Checklist para la entrega del Frontend

✅ Estructura organizada
(components/ pages/ hooks/ layouts/)

✅ Librería UI implementada
(react-bootstrap u otra)

✅ Routing con react-router
(Layout + páginas + navegación)

✅ Al menos 3 páginas principales funcionales

✅ Formularios controlados + validación con Zod

✅ Datos mock con useFetch
(3 estados: data / loading / error)

✅ *[Opcional]* useContext para estado transversal

✅ README.md en frontend/ con las notas de la entrega

🧰 **Herramientas:** useState, useFetch, custom Hooks, Zod, react-router, react-bootstrap, useContext



La Entrega Grupal 2 en concreto

RECORDATORIO
El **05/07/2026** también es la 2° instancia (menor puntaje) para la **Entrega de Actividades Individuales**

Trabajo Práctico, Fontend con React (Grupal)

El checklist previo es una guía para la entrega.

- **Cuándo**: 05/07/2026
- **Qué**: el frontend del TP grupal en el repo
- **Cómo se entrega**: úncamente por **GitHub** en su repositorio (rama **main**) y **notificando a los Docentes**
- **Qué incluir**: además del código completo funcionando, un README.md en la carpeta "frontend", dejando en él una explicación de la implementación y los comentarios que crean convenientes para la evaluación

⚠️ **Lo crítico:**

- Entregar en término sí o sí, ¡**no tiene recuperación!**
- **Avisar** a los docentes **por el canal del Grupo de Discord** una vez completada y subida al repositorio de GitHub
- Toda aclaración va dentro del repo (README), **NO en el mensaje de Discord**
- **Participación de todo el equipo**

❌ **Errores comunes** (evitar): dejar todo para el final, ramas sin mergear, README vacío, entrega que no funciona (probar bien el código final).

Cierre

Hoy cerramos el contenido de Frontend (Bloque 2)

Esta semana: terminar lo que quedó abierto y dejar la Entrega 2 lista

No hay actividades nuevas: la consigna es completar lo pendiente, Grupal e Individual. *Darle prioridad a lo Grupal.*

💬 **Consultas**: tienen el resto de la clase, aprovechen para revisar temas de la entrega con nosotros